



TALLER PRÁCTICO DE REACT - SISTEMA DE TIENDA ONLINE

Asignatura: Desarrollo Web con React

Objetivo: Desarrollar una tienda online funcional que consuma una API REST, implementando autenticación, manejo de estado, rutas protegidas y operaciones CRUD.

Fecha de entrega: _____

Forma de entrega: Repositorio en GitHub (enlace) + video de máximo 5 minutos demostrando el funcionamiento.

DESCRIPCIÓN DEL PROYECTO

Debes desarrollar una aplicación web en React que simule una tienda online. La aplicación debe consumir una API REST (puede ser desarrollada por ti con Node.js/Express, o utilizar una API simulada como JSON Server, o una API real como Firebase).

La aplicación tendrá dos roles principales:

Cliente:

- Registrarse
- Iniciar sesión
- Ver productos
- Agregar al carrito
- Realizar pedidos
- Ver historial de pedidos y detalles

Administrador:

- Iniciar sesión
- Ver listado de clientes registrados
- Crear nuevos administradores
- Ver todos los pedidos
- Aprobar o rechazar pedidos

REQUERIMIENTOS FUNCIONALES

1. AUTENTICACIÓN Y REGISTRO

- 1.1 Crear formulario de registro de clientes (nombre, email, contraseña, teléfono, dirección).
- 1.2 Crear formulario de inicio de sesión (email, contraseña) para ambos roles.
- 1.3 Al iniciar sesión, guardar el token y el rol del usuario en el almacenamiento local (localStorage o sessionStorage).
- 1.4 Redirigir al usuario según su rol: cliente → vista de tienda, administrador →



panel de administración.

1.5 Implementar rutas protegidas que solo permitan acceso a usuarios autenticados.

2. MÓDULO CLIENTE

2.1 Listado de productos: Mostrar todos los productos disponibles (imagen, nombre, precio, descripción corta).

2.2 Agregar al carrito: Botón en cada producto para añadirlo al carrito de compras.

2.3 Carrito de compras: Página donde se muestren los productos agregados con cantidad, subtotal y total.

2.4 Ajustar cantidades: Permitir aumentar/disminuir cantidad o eliminar productos del carrito.

2.5 Realizar pedido: Botón para confirmar el pedido, que lo registre en la API y vacíe el carrito.

2.6 Historial de pedidos: Página donde el cliente vea todos sus pedidos con fecha, total y estado.

2.7 Detalle del pedido: Al hacer clic en un pedido, mostrar los productos que lo componen (nombre, cantidad, precio unitario).

2.8 Estados del pedido: Mostrar el estado actual (Pendiente, Aprobado, Rechazado, Enviado, Entregado).

3. MÓDULO ADMINISTRADOR

3.1 Dashboard: Panel principal con resumen de clientes, pedidos y productos.

3.2 Listado de clientes: Ver todos los clientes registrados con su información básica.

3.3 Crear administradores: Formulario para registrar nuevos usuarios con rol de administrador.

3.4 Listado de pedidos: Ver todos los pedidos de todos los clientes, ordenados por fecha.

3.5 Aprobar/Rechazar pedidos: Botones para cambiar el estado de cada pedido (Aprobado/Rechazado).

3.6 Ver detalle del pedido: Al hacer clic en un pedido, mostrar los productos que lo componen y la información del cliente.

4. GESTIÓN DE PRODUCTOS (Opcional - Extra)

4.1 CRUD de productos: Crear, editar y eliminar productos (solo administradores).

REQUERIMIENTOS TÉCNICOS OBLIGATORIOS

1. React con Vite o Create React App - Proyecto configurado correctamente.
2. React Router DOM - Navegación entre páginas y rutas protegidas.



3. Context API o Redux (o Zustand) - Manejo del estado global (carrito, autenticación, usuarios).
4. Fetch API o Axios - Consumo de la API para todas las operaciones.
5. Hooks - useState, useEffect, useContext (o useDispatch/useSelector si usas Redux), useNavigate, etc.
6. Manejo de formularios - Formularios controlados con validación básica.
7. LocalStorage - Persistencia del token de autenticación.
8. Buenas prácticas - Componentes funcionales, carpetas organizadas, código limpio y comentado.
9. Estilos - Puedes usar CSS puro, CSS Modules, Tailwind, Bootstrap o cualquier framework de tu preferencia.

ESTRUCTURA DE CARPETAS SUGERIDA

```
src/  
├── api/  
│   └── apiClient.js  
├── components/  
│   ├── common/  
│   │   ├── Navbar.jsx  
│   │   ├── Footer.jsx  
│   │   └── ProtectedRoute.jsx  
│   ├── auth/  
│   │   ├── Login.jsx  
│   │   └── Register.jsx  
│   ├── client/  
│   │   ├── ProductList.jsx  
│   │   ├── ProductCard.jsx  
│   │   ├── Cart.jsx  
│   │   ├── CartItem.jsx  
│   │   ├── OrderHistory.jsx  
│   │   └── OrderDetail.jsx  
│   └── admin/  
│       ├── AdminDashboard.jsx  
│       ├── ClientList.jsx  
│       ├── CreateAdmin.jsx  
│       ├── OrderList.jsx  
│       └── OrderDetailAdmin.jsx  
├── context/  
│   ├── AuthContext.jsx  
│   ├── CartContext.jsx  
│   └── OrderContext.jsx  
└── hooks/
```



```
| └─ useAuth.js
| └─ pages/
|   └─ HomePage.jsx
|   └─ ShopPage.jsx
|   └─ CartPage.jsx
|   └─ OrdersPage.jsx
|   └─ AdminPage.jsx
|   └─ NotFoundPage.jsx
| └─ styles/
|   └─ globals.css
| └─ utils/
|   └─ helpers.js
| └─ App.jsx
| └─ main.jsx
```

API - ENDPOINTS SUGERIDOS

Autenticación:

- POST /api/auth/register - Registrar cliente
- POST /api/auth/login - Iniciar sesión (devuelve token + rol)
- POST /api/auth/register-admin - Registrar administrador (solo admin)

Productos:

- GET /api/products - Obtener todos los productos
- GET /api/products/:id - Obtener un producto
- POST /api/products - Crear producto (solo admin)
- PUT /api/products/:id - Editar producto (solo admin)
- DELETE /api/products/:id - Eliminar producto (solo admin)

Pedidos:

- POST /api/orders - Crear un pedido (cliente)
- GET /api/orders/user - Obtener pedidos del cliente autenticado
- GET /api/orders - Obtener todos los pedidos (solo admin)
- GET /api/orders/:id - Obtener detalle de un pedido
- PUT /api/orders/:id/status - Cambiar estado del pedido (solo admin)

Usuarios:

- GET /api/users - Obtener todos los usuarios (solo admin)
- GET /api/users/:id - Obtener detalle de usuario (solo admin)

DISEÑO Y UX

1. La interfaz debe ser responsive (adaptarse a móvil, tablet y escritorio).
2. Utilizar una paleta de colores consistente y profesional.
3. Mostrar indicadores de carga (spinners o skeletons) mientras se cargan los datos.
4. Mostrar mensajes de éxito o error (toasts o alerts) al realizar operaciones.



5. El carrito debe mostrar un contador en el navbar con la cantidad de productos.
6. Los botones deben tener estados (hover, active, disabled) bien definidos.

ENTREGABLES

1. Repositorio en GitHub con el código fuente completo.
2. README.md con:
 - Descripción del proyecto
 - Instrucciones de instalación y ejecución
 - Credenciales de prueba (cliente y administrador)
 - Enlace a la API utilizada (si es pública)
3. Video de demostración (máximo 5 minutos) mostrando todas las funcionalidades.
4. Despliegue (opcional - extra): La aplicación desplegada en Vercel, Netlify o similar.

EXTRAS (Puntos adicionales)

1. Desplegar la aplicación en Vercel o Netlify.
2. Implementar filtros de búsqueda de productos.
3. Implementar paginación en listados.
4. CRUD completo de productos (crear, editar, eliminar).
5. Subir imágenes de productos.
6. Implementar Dark Mode.
7. Pruebas unitarias con Jest y React Testing Library.

RECOMENDACIONES FINALES

1. Planifica antes de codificar: Dibuja el flujo de la aplicación y las relaciones entre componentes.
2. Versiona tu código: Haz commits frecuentes y descriptivos.
3. Prueba cada funcionalidad antes de pasar a la siguiente.
4. No dejes todo para el final: Avanza paso a paso.
5. Pide ayuda si te quedas bloqueado, pero documenta el problema y cómo lo solucionaste.
6. Diviértete y aprovecha para explorar nuevas herramientas y librerías.

¡Éxito con tu taller! 🚀